

Detección temprana de pedidos anómalos de clientes mediante predicción de referencia y árboles de decisión

Proyecto de Fin de Carrera I — Entregable 1

Sefora Cieza Lujan
Universidad de Ingeniería y Tecnología (UTEC)
`sefora.cieza@utec.edu.pe`

6 de septiembre de 2026

Resumen

Bimbo despacha a 61 tiendas Bembos lo que el sistema de compras del cliente le indica, sin ninguna validación previa de que esa cantidad tenga sentido. Cuando la tienda se equivoca al registrar el pedido, el error se convierte en producto despachado. Este informe delimita ese problema, lo cuantifica sobre 10 374 líneas de pedido vigentes y propone automatizar la verificación: estimar cuánto debería pedir cada tienda a partir de su propio historial y comparar esa cifra con lo que pidió de verdad, dejando que un árbol de decisión clasifique la diferencia y explique por qué la marcó. El análisis exploratorio muestra que el método reencuentra por sí solo el caso que el área ya había detectado a mano, genera unas dos alertas por semana y tiene un riesgo identificado: confundir un feriado con un error.

Índice

1. El problema	2
1.1. Cómo funciona hoy la operación	2
1.2. Lo que dijeron y lo que realmente pasa	3
1.3. Por qué pasa: cinco porqués	3
1.4. Ishikawa	3
1.5. La evidencia	3
1.6. El conjunto de datos	5
1.6.1. Diccionario de columnas	6
1.7. Qué dicen los datos	7
1.7.1. Las líneas anuladas y el doble pedido	7
1.7.2. El método encuentra el caso que ya conocíamos	7
1.7.3. Cuánto producto se despacha de más	8
1.7.4. El feriado también levanta los pedidos	8
1.7.5. Separar la campaña del error sin preguntarle a nadie	8
1.7.6. Los parámetros del método	10

1.7.7. Cuántas alertas serían	10
1.8. A quién le duele	11
1.9. Los stakeholders y quién decide	11
2. La solución	11
2.1. La propuesta	11
2.2. Por qué el proyecto se limita a estos datos	12
2.3. Cómo funciona	13
2.4. Con qué variables decide el árbol	13
2.5. De dónde sale la etiqueta	13
2.6. Contra qué se compara	14
2.7. El pedido que no llegó	14
2.8. El software	14
2.8.1. Cargar el archivo	15
2.8.2. Revisar las alertas	15
2.8.3. Ver lo que no llegó	15
2.8.4. Exportar	16
2.8.5. Cómo está construido	16
2.9. Objetivo	16
2.10. Objetivos específicos	16
3. Alcance y verificación	17
3.1. Qué entra y qué no	17
3.2. Restricciones, supuestos e incertidumbres	17
3.3. Qué se va a medir	18
3.4. Indicadores	18
3.5. Criterios de aceptación del prototipo	18
3.6. Viabilidad	19
3.7. Matriz de riesgos	20
3.8. Plan de trabajo	20
4. Retroalimentación incorporada	21
5. Estado actual del proyecto	23

1. El problema

1.1. Cómo funciona hoy la operación

Bimbo abastece de pan de hamburguesa a las tiendas Bembos, operadas por NGR y EyH. Cada tienda calcula lo que va a necesitar, el área de compras consolida esas necesidades y emite una orden de compra en el ERP. El proveedor produce y despacha contra esa orden, y factura a 60 días.

Todo el proceso descansa en un supuesto que nadie verifica: que la cantidad escrita en la orden es la cantidad correcta. Si una tienda registra 432 unidades donde quería 108, el sistema no lo cuestiona. El único filtro es que alguien del proveedor abra el archivo, note algo raro y escriba un correo preguntando.

1.2. Lo que dijeron y lo que realmente pasa

La queja llegó así, con estas palabras: «*observo cantidades altas en pedido de las tres presentaciones, como ejemplo de doble pedido de algunas tiendas*». Conviene separar tres cosas que suelen confundirse.

Tabla 1: Separación entre lo que se ve, lo que falla y la solución precipitada.

Síntoma	Problema	Solución que aparece primero
Llegan pedidos con cantidades muy por encima de lo habitual, y a veces la misma línea repetida	No existe ningún control que compare la cantidad pedida contra el comportamiento histórico de esa tienda antes de que el pedido se despache	«Que alguien revise el archivo todos los días» o «que el ERP tenga un tope»

La diferencia importa. Revisar el archivo a mano ya se hace y no alcanza; poner un tope fijo trata igual a una tienda de centro comercial y a una de barrio. El problema no es que falte esfuerzo, es que falta un criterio automático que use el historial de cada tienda.

1.3. Por qué pasa: cinco porqués

1. **¿Por qué se despachan pedidos con cantidades erróneas?** Porque el proveedor produce contra lo que dice la orden de compra.
2. **¿Por qué produce contra una orden que puede estar mal?** Porque no tiene forma de saber que lo está antes de producir.
3. **¿Por qué no tiene forma de saberlo?** Porque la única revisión es visual y depende de que un analista reconozca el patrón en un archivo de cientos de líneas.
4. **¿Por qué la revisión es visual?** Porque no hay un valor de referencia calculado contra el cual comparar cada línea.
5. **¿Por qué no hay valor de referencia?** Porque el historial de pedidos por tienda y producto existe en el sistema pero nunca se ha usado para construir uno. **Esa es la causa raíz.**

Si se elimina esa causa, el efecto desaparece: la comparación deja de depender de que alguien tenga un buen día.

1.4. Ishikawa

Los cinco porqués siguen una sola cadena. El diagrama de causa-efecto de la Figura 1 abre esa cadena en las seis familias de causas que suelen revisarse en una operación, para comprobar que no queda ninguna fuera.

1.5. La evidencia

Tres correos de la operación sostienen lo anterior. El primero (Figura 2) es el mensaje interno que originó la revisión: advierte cantidades altas en las tres presentaciones, con doble pedido en varias tiendas.

Los otros dos muestran el control actual funcionando y fallando. En el segundo, el proveedor avisa que la tienda de ATE tiene un pedido *«muy alto en comparación a su despacho*

del 1 de julio al 11 de septiembre de 2026.

Tabla 2: Muestra de las primeras filas del archivo.

Fe. entrega	Orden	Pos.	Material	Tienda	Cantidad	Valor S/
11-09-2026	4500730889	10	201985	BB TDA MONTERRICO	180	125,40
11-09-2026	4500730889	20	201986	BB TDA MONTERRICO	330	193,60
11-09-2026	4500730889	30	201987	BB TDA MONTERRICO	480	238,40
11-09-2026	4500730889	40	201985	BB TDA AURORA	144	100,32
11-09-2026	4500730889	50	201986	BB TDA AURORA	420	246,40
11-09-2026	4500730889	60	201987	BB TDA AURORA	420	208,60

1.6.1. Diccionario de columnas

De las 28 columnas, 12 aportan información al análisis, 12 son constantes o administrativas y 4 vienen completamente vacías. La Tabla 3 las describe.

Tabla 3: Columnas del archivo, su contenido y su uso en el proyecto.

Columna	Contenido	Valores distintos	Uso
Fecha documento	Día en que se registró la orden	30	Contexto
Fe. Entrega	Día comprometido de entrega	68	Clave
Documento compras	Número de orden de compra	156	Agrupación
Posición	Número de línea dentro de la orden	197	Duplicidad
Material	Código del producto	3	Clave
Texto breve	Descripción del producto	3	Reporte
Tienda	Nombre de la tienda solicitante	61	Clave
Centro	Código de la tienda	61	Equivale a Tienda
Cantidad de pedido	Unidades solicitadas	115	Variable objetivo
Cantidad en UMA	Igual a la anterior en este extracto	115	Verificación
Por entregar (cantidad)	Pendiente de despacho	61	Contexto
Valor neto de pedido	Importe de la línea en soles	196	Costo del exceso
Precio neto	Precio de referencia del producto	59	Contexto
Indicador de borrado	Marca L si la línea fue eliminada	1	Filtro
Creado por	Usuario que registró la orden	23	Contexto
Nombre del proveedor	Razón social del proveedor	7	Filtro
N.º Factura, Ref. Sunat, Fecha Pago	Datos de facturación, presentes en 6 926 filas	—	No se usan
Unidad medida pedido, Unidad de almacén, Moneda, Grupo de artículos, Cantidad base	Constantes en todo el archivo	1	No aportan
Historial pedido, Número de necesidad, Num. EM o AC, Tipo de imputación	Columnas vacías en las 10 772 filas	0	No se usan

Tres observaciones sobre la calidad del archivo condicionan el análisis. La unidad de medida es única (UN) en todas las filas, de modo que no hay que convertir entre bolsas, bandejas y unidades dentro de este extracto. El campo de facturación está presente solo en 6 926 filas, porque las órdenes más recientes aún no se facturan, así que no sirve para

verificar lo entregado. Y el indicador de borrado obliga a un filtro previo que se explica en la Sección 1.7.1.

Con los tres campos clave, que son tienda, material y fecha de entrega, más la cantidad, el archivo se reorganiza en 180 series semanales de tienda y producto, que son la materia prima del método. De ellas, 176 tienen suficiente historial previo para poder evaluarse, lo que deja 1 554 observaciones de tienda, producto y semana sobre las que se mide todo lo que sigue.

1.7. Qué dicen los datos

Los correos describen el fenómeno; el extracto del sistema de compras permite medirlo.

Conviene precisar el alcance de lo que sigue. Las cifras y figuras de esta sección provienen de un análisis exploratorio **ya ejecutado** sobre el extracto real, con un cuaderno en Python que se vuelve a correr sobre cualquier archivo del mismo formato. No son estimaciones ni ejemplos ilustrativos. Lo que todavía no existe es el software descrito en la Sección 2.8: el método está probado, la herramienta que lo pone en manos de quien revisa está por construirse. Los resultados de esta sección son, por tanto, la evidencia de que el enfoque funciona sobre los datos históricos, y no una medición del sistema en operación.

1.7.1. Las líneas anuladas y el doble pedido

Entre las columnas del archivo hay una llamada *indicador de borrado*. Está vacía casi siempre, pero en 398 líneas dice L, la marca que usa el sistema cuando una posición se elimina. Vale la pena mirarlas, porque cambian la cuenta.

De esas 398 líneas anuladas, 385 tienen al menos una línea activa para la misma tienda, el mismo producto y la misma fecha de entrega. No son pedidos adicionales: son pedidos que se borraron y se volvieron a cargar. Al comparar cada anulada con su reemplazo, en la enorme mayoría de los casos la cantidad es idéntica, es decir que se volvió a cargar exactamente lo mismo, y solo en 27 la cantidad cambió, con la versión definitiva mayor en 19 casos y menor en 8.

Esto obliga a trabajar con las 10 374 líneas vigentes, y corrige una cuenta que sin ese filtro sale mal. Si se dejan las anuladas junto con sus reemplazos, el archivo aparenta tener cientos de pedidos repetidos con la misma cantidad, cuando en realidad se está contando dos veces el mismo pedido. Descontándolas quedan **54 casos de cantidad idéntica repetida, que suman 119 líneas**, y 176 combinaciones de tienda, producto y fecha con más de una línea. Ese es el volumen real del doble pedido, y para encontrarlo no hace falta ningún modelo: alcanza con una consulta.

Las 27 líneas donde la corrección cambió la cantidad sirven además como etiquetas: son pedidos que alguien registró de una forma y después rectificó.

1.7.2. El método encuentra el caso que ya conocíamos

Se calculó, para cada tienda y producto, la mediana de las cuatro semanas previas como valor de referencia $\hat{q}_t$, y la diferencia relativa contra el pedido real, $e_t = (q_t - \hat{q}_t)/\hat{q}_t$. La Figura 5 muestra qué pasa con la tienda de ATE en pan grande.

Esa semana marcada es el pedido del sábado 22 de agosto, el mismo que motivó el correo de la Figura 3. El cálculo llegó a la misma conclusión que la analista, sin que nadie le indicara qué buscar. Es la mejor señal de que el criterio automático coincide con el juicio experto que hoy sostiene la verificación.

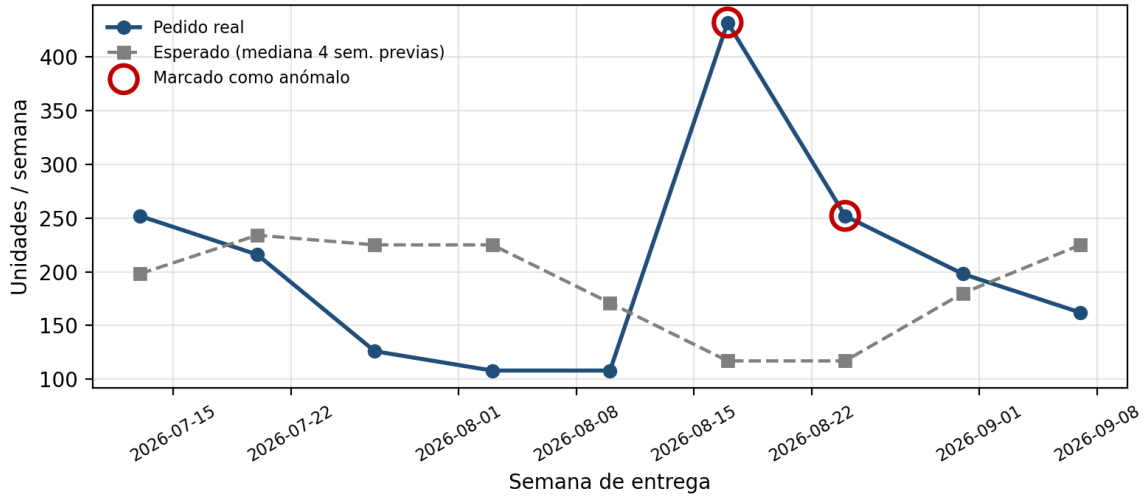


Figura 5: Tienda ATE, pan grande: pedido real contra valor de referencia. La semana del 17 de agosto pide 432 unidades donde se esperaban 117.

1.7.3. Cuánto producto se despacha de más

El impacto no está en la merma de la tienda sino en el propio despacho: se produce, se transporta y se factura una cantidad que el cliente no pidió realmente. Con el precio unitario implícito en cada línea (mediana S/ 0,59) se puede poner número a ese exceso.

Tabla 4: Exceso despachado sobre el valor de referencia, umbral $\tau = 0,8$.

Casos detectados en 9 semanas evaluables	17
Exceso acumulado (unidades)	11 200
Exceso acumulado (S/)	6 625
Exceso mensual estimado (S/)	3 187

Alrededor de **S/ 3 200 al mes** en producto despachado por encima de lo que el comportamiento del cliente permitía anticipar. La cifra descuenta el movimiento común de la cadena, según el procedimiento de la Sección 1.7.5, y excluye las líneas anuladas.

1.7.4. El feriado también levanta los pedidos

No todas las alzas son errores. La semana del 27 de julio, Fiestas Patrias, llega al 12,4 % de series con alza superior al 50 %, frente a un nivel de fondo cercano al 2 %. Una regla de umbral marcaría veinte tiendas esa semana, todas correctas, y el analista dejaría de mirar las alertas en poco tiempo. Distinguir el feriado del error exige una pregunta que ningún umbral sobre una sola variable puede responder: si la tienda subió sola o subió junto con todas las demás. La Sección 1.7.5 la responde con los datos.

1.7.5. Separar la campaña del error sin preguntarle a nadie

Un error de digitación de una tienda no mueve el promedio de las otras 175 series; un feriado sí. Sobre esa diferencia se puede construir un criterio que distinga ambos casos con los datos disponibles, sin depender de que alguien recuerde qué ocurrió una semana determinada.

Se define un **factor común semanal** a_t como la mediana del residuo de todas las series esa semana, y se descompone

$$e_t = \underbrace{a_t}_{\text{lo que le pasó a todo el universo}} + \underbrace{u_t}_{\text{lo que le pasó solo a esta tienda}}.$$

La decisión de anomalía se toma sobre u_t .

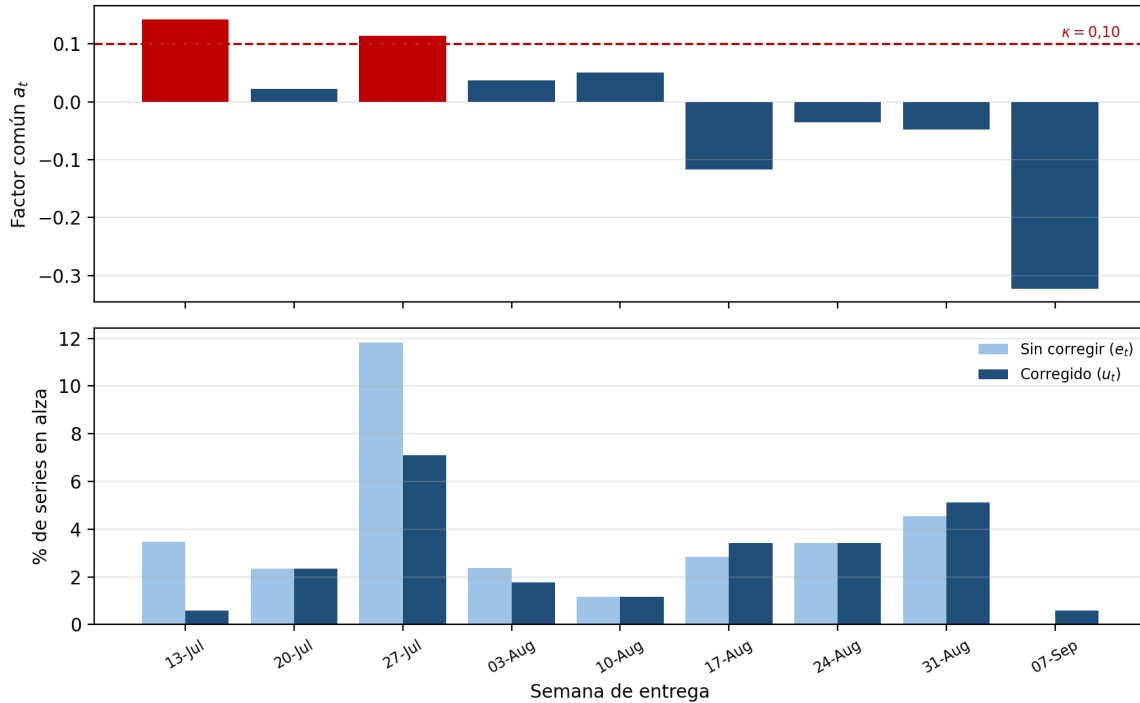


Figura 6: Arriba, el factor común semanal. Las dos semanas de julio que lo superan corresponden a movimientos de toda la cadena. Abajo, el porcentaje de series en alza antes y después de descontar ese factor.

La semana con mayor movimiento conjunto es la del 27 de julio, Fiestas Patrias, con un factor común de +0,114. Tres señales permiten separarla del caso de ATE, que es un error confirmado por correo.

Tabla 5: Firma de una semana de campaña frente a la de un error de captura.

Señal	27-jul (Fiestas Patrias)	17-ago (caso ATE)
Factor común a_t	+0,114	-0,117
Series en alza, sin corregir	12,4 %	2,8 %
Series en alza, corregido	7,7 %	3,4 %
Tiendas que suben en las 3 presentaciones	8 de 57	0 de 60
Residuo mediano la semana siguiente	+0,01	+0,71

Las dos últimas filas son las que más separan un caso del otro. En Fiestas Patrias suben las tres presentaciones a la vez en varias tiendas y el efecto se agota la semana siguiente, que vuelve a su nivel normal. En ATE ninguna tienda movió las tres presentaciones y el residuo sigue alto siete días después, que es el comportamiento esperable de un error arrastrado y no de un pico de consumo.

1.7.6. Los parámetros del método

El procedimiento no depende de un juicio externo, pero sí de cinco números que conviene nombrar y ajustar con validación temporal en lugar de fijarlos a ojo.

Tabla 6: Hiperparámetros del detector.

Símbolo	Qué controla	Valor inicial	Cómo se ajusta
k	Semanas de historial para la mediana de referencia	4	Error de predicción contra persistencia
τ	Cuánto tiene que desviarse una tienda por su cuenta (u_t) para marcar la línea	0,8	Precisión contra carga de alertas
κ	Factor común a partir del cual la semana se declara de campaña	0,10	Semanas conocidas de feriado
σ	Presentaciones que deben subir juntas para atenuar la alerta	2 de 3	Casos marcados como error
ρ	Reversión esperada la semana siguiente para confirmar campaña	0,00	Histórico de semanas de feriado

Dejar la pregunta resuelta como parámetro y no como consulta tiene una ventaja sobre preguntar: el método vuelve a clasificar correctamente la próxima semana atípica sin que nadie tenga que recordar qué pasó en esta.

1.7.7. Cuántas alertas serían

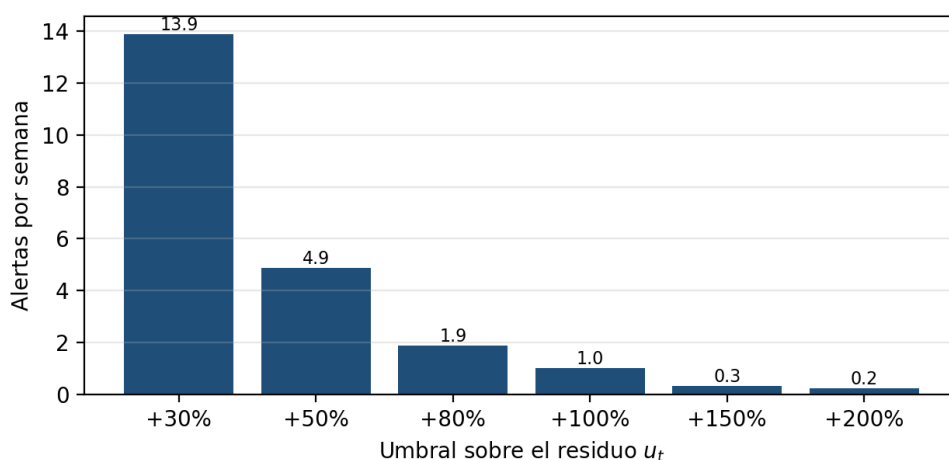


Figura 7: Alertas semanales esperadas según el umbral sobre u_t . Con $\tau = 0,8$ salen unas 2 por semana.

Dos alertas por semana caben sin problema en la ventana de 30 minutos que ya está pactada. Un umbral más bajo detectaría más errores, pero a costa de un volumen que nadie va a revisar, y una alerta que no se revisa vale lo mismo que ninguna.

1.8. A quién le duele

Tabla 7: Matriz impacto–interesados.

Interesado	Cómo le afecta hoy	Magnitud al mes	Evidencia
Proveedor (Bimbo)	Produce, transporta y factura una cantidad distinta de la que el cliente realmente necesitaba	$\approx$ S/ 3 200	Tabla 4
Planeamiento (Bimbo)	Revisa el archivo línea por línea y redacta los correos de confirmación	17 casos por revisar cada 9 semanas	Figura 3
Compras (NGR)	Recibe la alerta, verifica con la tienda y rectifica la orden	27 líneas rectificadas en el extracto	Indicador de borrado
Tiendas	Reciben una cantidad que no corresponde a su pedido	54 casos de cantidad repetida	Sección 1.7.1
Producción (Bimbo)	Planifica sobre una señal de demanda distorsionada	Indirecta	Efecto conocido en cadenas de suministro [9]

1.9. Los stakeholders y quién decide

Tabla 8: Ubicación de los interesados según poder e interés.

Interesado	Cuadrante	Qué necesita	Frecuencia
Jefatura de planeamiento (Bimbo)	Alto poder, alto interés	Que las alertas bajen su carga manual y sean defendibles ante el cliente	Semanal
Analista de planeamiento	Bajo poder formal, alto interés	Que el reporte llegue antes del cierre del pedido y no le sume trabajo	Semanal
Compras (NGR)	Alto poder, interés medio	Que la alerta no frene un pedido legítimo	Quincenal
Tiendas	Bajo poder, alto interés	No quedar desabastecidas por una alerta mal disparada	Por excepción
TI del cliente	Alto poder, bajo interés	Que nada de esto exija tocar el ERP en esta etapa	Por hito

Responsable del resultado: Sefora Cieza Lujan, autora del proyecto. Lo ejecuta de principio a fin: la preparación de los datos, el marcado de los casos, el modelo y el software. La jefatura recibe el resultado al cierre y el asesor aprueba el entregable académico, pero ninguna etapa intermedia depende de que un tercero responda.

2. La solución

2.1. La propuesta

La solución es un software de validación de pedidos. Quien revisa los pedidos carga el Excel que ya exporta del sistema de compras y el programa devuelve otro con los productos

Tabla 9: Responsabilidades sobre los entregables (RACI).

Actividad	Practicante	Asesor	Jefatura	Cliente
Preparación y limpieza de datos	R/A	I	I	I
Marcado de los casos que fueron error	R/A	C	I	I
Construcción y validación del modelo	R/A	C	I	I
Calibración del umbral de alerta	R/A	C	I	I
Construcción del software	R/A	I	I	I
Presentación de resultados a la empresa	R	I	A	I
Aprobación del entregable académico	R	A	I	I

cuyo pedido no coincide con lo que el historial del cliente permitía esperar. Un archivo entra, un archivo sale, y en lugar de revisar miles de líneas una por una el analista revisa las pocas que quedan marcadas. El programa no reemplaza la decisión: para cada producto marcado indica cuánto pidió la tienda, cuánto se esperaba según sus semanas anteriores y por qué se marcó, de modo que esa misma fila sirva para reclamarle al cliente. La confirmación final sigue siendo humana.

Hay dos caminos más obvios que este y ninguno funciona acá. Un tope fijo de cantidad en el ERP trataría igual a una tienda de centro comercial que a una de barrio, y exigiría modificar un sistema que no controlamos. Pedir a las tiendas que confirmen cada pedido por segunda vez agrega un paso diario a un flujo que ya va apurado y no aprende nada: dentro de un año el problema sigue igual. Calcular el valor esperado desde el propio historial evita ambos inconvenientes, porque cada tienda se compara consigo misma y el criterio mejora conforme se acumulan semanas. Es además lo que el área ya hace a mano cuando compara un pedido contra el despacho anterior, así que automatizarlo no le pide a nadie cambiar de forma de pensar.

Dentro de ese enfoque, el árbol de decisión [3, 14] se eligió por una razón práctica antes que estadística. Para reclamarle a un cliente hace falta poder decirle por qué: «según sus últimas cuatro semanas esperábamos 117 unidades, usted pidió 432, y además hay una línea duplicada». Un modelo que solo entrega un puntaje traslada el problema de confianza en lugar de resolverlo.

La variable que no se mueve es la **precisión de las alertas**. Un sistema que marca de más se deja de mirar, que es el fracaso ya documentado en la Figura 4. Lo primero que cede es la **cobertura**: se prefiere dejar pasar algunos errores pequeños antes que inundar de avisos al analista. También cede el **alcance de la integración**, porque el software trabaja sobre el archivo exportado y no toca el ERP del cliente.

2.2. Por qué el proyecto se limita a estos datos

El trabajo se restringe al pan de hamburguesa que Bimbo despacha a las tiendas Bembos operadas por NGR y EyH: tres productos, 61 tiendas y 11 semanas de historial. No se abordan otros proveedores, categorías ni cadenas.

La restricción es deliberada y mejora el resultado. Las tres presentaciones comparten unidad de medida, ciclo de reposición y comportamiento estacional, de modo que el valor esperado de una tienda es comparable con el de otra y el factor común semanal tiene sentido. Al ampliar a categorías con rotación distinta ese supuesto se rompe y habría que estimar un modelo por familia con menos datos cada uno. Trabajar sobre un universo cerrado permite

además verificar contra hechos conocidos: el caso de la tienda de ATE está documentado por correo y sirve de comprobación de que el método detecta lo que debe, contraste que no existiría en un conjunto disperso de proveedores. La generalización a otras categorías queda como continuación natural, una vez validado el método en este universo.

2.3. Cómo funciona

El sistema corre sobre el archivo de pedidos antes del despacho y tiene dos partes. La primera calcula, con el historial hasta la semana anterior, cuánto debería pedir cada tienda de cada producto. La segunda compara ese número con el pedido real y decide si la diferencia es normal, usando un árbol de decisión [3, 14] sobre el residuo $e_t = (q_t - \hat{q}_t) / \max(\hat{q}_t, 1)$ y las variables de la Tabla 10.

Para el valor esperado $\hat{q}_t$ se probarán tres opciones, de la más simple a la más elaborada: el pedido de la semana anterior, $\hat{q}_t = q_{t-1}$, que es lo que hace hoy la analista y por eso es la referencia obligada; la mediana de las cuatro semanas previas, que aguanta que el propio historial tenga pedidos equivocados, cosa que el promedio no hace; y suavizamiento exponencial con ajuste por día de la semana [7], si el histórico da para tanto. Hay un detalle que puede arruinar el cálculo si se pasa por alto: el historial contiene los errores que se quieren detectar, así que la referencia sale inflada. Por eso se usan estimadores robustos y se recalcula excluyendo lo que la primera pasada marcó.

2.4. Con qué variables decide el árbol

Tabla 10: Variables del clasificador.

Variable	Qué mide
e_t	Diferencia relativa entre lo pedido y lo esperado.
z_{rob}	Cuántas desviaciones robustas se aleja el pedido de la mediana de su serie.
Δ_{prev}	Variación contra el pedido de la semana anterior.
d_{dup}	Si existe otra línea con la misma tienda, producto y cantidad en la misma fecha.
ρ_{mix}	Peso del producto dentro del pedido total de la tienda, comparado con su peso histórico.
c_{sku}	Si las tres presentaciones se movieron juntas. Cuando suben todas a la vez suele ser demanda real.
a_t	Mediana del residuo de todas las series esa semana. Separa el feriado del error de una tienda sola.
dow	Día de la semana de entrega.

Las variables c_{sku} y a_t salen del hallazgo de la Sección 1.7.5 y son las que un umbral simple no puede usar.

2.5. De dónde sale la etiqueta

El extracto no trae un campo que diga «este pedido estuvo mal», así que hay que construirlo en dos pasos. Primero se marcan como candidatos los casos que el área ya reconoce: duplicidad exacta, residuo por encima del umbral acordado, diferencia entre lo pedido y lo consumido. Después se revisan uno por uno y se marcan como error o como normales.

Hay dos fuentes adicionales de etiquetas. Los correos de alerta guardan casos ya confirmados por escrito. Y de las 398 líneas anuladas, 27 fueron reemplazadas por una cantidad distinta: son pedidos que alguien registró de una forma y luego rectificó, es decir, errores reconocidos por la propia operación sin costo de revisión adicional.

2.6. Contra qué se compara

El árbol se mide contra tres referencias. Un umbral sobre el pedido anterior, que reproduce el control manual actual y es el piso obligado; un umbral sobre z_{rob} , como regla estadística simple; y un Isolation Forest [10], que no necesita etiquetas y sirve para verificar si el criterio de etiquetado tiene sentido. El árbol se implementa en `scikit-learn` con profundidad limitada [12], y se usa gradient boosting [5] solo como techo de referencia, para saber cuánta exactitud cuesta la interpretabilidad; si se usa, se acompaña de valores de Shapley [11].

Si el árbol no le gana a los umbrales, hay que decirlo: que el problema se resuelva con una regla simple es un resultado válido, no un fracaso. El marco general de la tarea y sus métricas sigue a Chandola et al. [4], Hodge y Austin [6] y Aggarwal [1]; el proceso de trabajo sigue las fases de CRISP-DM [15] dentro del marco del PMBOK [13] y la adaptación de la forma de trabajo de Ambler y Lines [2].

2.7. El pedido que no llegó

Un pedido con cantidad equivocada se ve en el archivo; uno que nunca se emitió, no. Para el proveedor la segunda situación puede costar más: la tienda se queda sin producto, la venta no ocurre y nadie se entera hasta que alguien reclama.

El mismo cálculo que estima cuánto debería pedir una tienda detecta esos casos. Si una serie tienda–producto tiene valor esperado positivo para la semana en curso y en el archivo no aparece ninguna línea, la ausencia se reporta igual que una desviación de cantidad.

Las 180 series de tienda y producto por 11 semanas dan 1980 combinaciones posibles, de las cuales 1913 tienen pedido. Faltan 67, y **21 son huecos intermedios**: semanas sin pedido entre semanas con pedido, repartidos en 7 series de tres tiendas. No están dispersos en el tiempo, sino concentrados entre el 20 de julio y el 10 de agosto, y las tres tiendas dejan de pedir y vuelven en fechas próximas.

Ese patrón importa para el diseño. Una ausencia no es por sí sola un error: puede ser una pausa con motivo, y varias tiendas que se detienen a la vez en la misma ventana apuntan a una causa compartida antes que a olvidos independientes. Distinguir una cosa de la otra exige el mismo contexto histórico que el resto del método y no un criterio aparte.

El software reporta las ausencias en una hoja separada del archivo de salida, con la tienda, el producto, la cantidad esperada y cuántas semanas lleva sin pedir, y deja la interpretación a quien conoce la operación.

2.8. El software

El entregable final es un software de validación de pedidos que funciona de la forma más simple posible: **entra un Excel y sale un Excel**. Quien revisa los pedidos carga el mismo archivo que hoy exporta del sistema de compras, sin transformaciones previas, y el programa devuelve otro con los productos cuyo pedido no coincide con lo que su historial permitía esperar. Lo que hoy significa recorrer miles de líneas a la vista pasa a ser revisar las pocas que quedan marcadas.

Entre esos dos extremos hay cuatro pantallas.

2.8.1. Cargar el archivo

El usuario arrastra el Excel y el programa revisa que sirva antes de procesarlo: que estén las columnas necesarias, que las fechas sean fechas y las cantidades números, y que el archivo corresponda al proveedor esperado. Si algo falta, lo dice con nombre propio, «no encuentro la columna Fe.Entrega», en lugar de fallar sin explicación. También avisa cuántas líneas anuladas encontró y cuántas va a analizar, para que el usuario sepa sobre qué está trabajando.

2.8.2. Revisar las alertas

La pantalla principal es una tabla con una fila por producto marcado, ordenada por valor en riesgo, con las columnas de la Tabla 11 y filtros por tienda, producto y nivel de alerta.

Tabla 11: Columnas del resultado.

Columna	Contenido
Tienda	Nombre y código de la tienda.
Material y descripción	Código y texto breve del producto.
Fecha de entrega	Fecha comprometida de la línea.
Cantidad pedida / esperada	Lo solicitado y lo que el historial anticipaba.
Desviación	Diferencia porcentual entre ambas.
Motivo	Cantidad fuera de rango, duplicidad, o ambas.
Nivel	Alta, media o baja, según cuánto supere el umbral.
Valor en riesgo	Exceso en unidades por el precio unitario.
Últimas semanas	Pedidos previos de esa tienda y producto.

Al hacer clic en una fila se abre una **ficha de detalle** con lo que el analista necesita para decidir sin salir del programa: el gráfico de las últimas semanas de esa tienda y producto con el pedido actual resaltado, las otras dos presentaciones de la misma tienda esa semana, porque si subieron las tres es más probable que sea demanda real, y la regla exacta que disparó la alerta escrita en palabras.

Desde la misma ficha el analista marca la alerta como *confirmada* o *descartada*, y en el segundo caso escribe por qué. Ese registro de decisiones tiene un efecto que va más allá del orden: cada semana de uso genera ejemplos etiquetados sin trabajo adicional, de modo que el modelo puede reentrenarse con casos reales en lugar de depender de una única muestra revisada al inicio. La herramienta produce, usándose, el insumo que necesita para mejorar.

2.8.3. Ver lo que no llegó

Una segunda pestaña muestra las ausencias de pedido descritas en la Sección 2.7: tiendas y productos que debían pedir esa semana y no aparecen en el archivo, con la cantidad esperada y cuántas semanas llevan sin pedir. En la cabecera aparece además el aviso de campaña cuando el movimiento común de la semana supera el umbral κ , para que nadie interprete como error un alza que afectó a toda la cadena.

2.8.4. Exportar

El botón de descarga genera el Excel de salida con las alertas, las ausencias y las decisiones tomadas. Es el archivo que se adjunta al correo del cliente, y reemplaza el que hoy se arma a mano.

Una pantalla de configuración permite ajustar el umbral de alerta y la ventana de historial sin tocar el código, porque esos valores se calibran con el uso y no deberían requerir a un programador cada vez.

2.8.5. Cómo está construido

Se construye en Python con `pandas` y `scikit-learn` [12], y la interfaz se arma con Streamlit, que convierte un script de análisis en una aplicación con carga de archivos, tablas, filtros y gráficos sin escribir código de frontend. Corre en el equipo del usuario. Internamente encadena cinco pasos: lee y valida el archivo, agrupa por tienda–producto–semana, calcula el valor esperado con la mediana de las k semanas previas, separa el movimiento común de la cadena del propio de cada tienda, y aplica el árbol entrenado devolviendo la marca junto con la regla que la produjo. El modelo entrenado se guarda en disco, así que no reentrena en cada ejecución y responde en segundos. No requiere servidor, base de datos ni conexión con el ERP: trabaja sobre el archivo, igual que hoy trabaja la analista.

2.9. Objetivo

General. Construir y validar un sistema que estime el pedido esperado por tienda y producto a partir del historial, y que clasifique con un árbol de decisión las desviaciones frente al pedido real, alcanzando una precisión de al menos 70 % sobre los casos marcados como error, con no más de cinco alertas semanales, dentro de las 12 semanas del proyecto.

Tabla 12: El objetivo, letra por letra.

S	Detectar pedidos anómalos por tienda y producto antes del despacho, con la regla que justifica cada alerta.
M	Precisión $\geq 70\%$ y exhaustividad $\geq 80\%$ sobre el conjunto de casos marcados; entre 2 y 5 alertas por semana.
A	Los datos existen (11 semanas, 61 tiendas), las herramientas son de uso libre y el volumen corre en un equipo estándar.
R	Ataca un costo de alrededor de S/ 3200 al mes y una tarea manual que hoy no escala.
T	Semana 12 del proyecto, con validación temporal sobre las últimas tres semanas del extracto.

2.10. Objetivos específicos

1. Limpiar y consolidar el extracto en una tabla tienda–producto–semana.
2. Implementar el cálculo del valor esperado y compararlo contra la persistencia.
3. Marcar los casos que fueron error a partir del historial disponible.
4. Entrenar el árbol y las líneas base, y evaluarlos con validación temporal.
5. Construir el software que recibe el Excel de pedidos y devuelve los productos anómalos con su descripción, la desviación y el motivo.

3. Alcance y verificación

3.1. Qué entra y qué no

Tabla 13: Frontera del proyecto.

Dentro	Fuera
Limpieza y consolidación del extracto	Pronóstico de demanda para planificar producción
Cálculo del pedido esperado por tienda y producto	Integración dentro del ERP del cliente
Variables de desviación y contexto	Optimización de rutas, inventario o producción
Marcado de los casos que fueron error	Corrección automática del pedido
Árbol de decisión y líneas base, con validación temporal	Otros proveedores, otras categorías y otras cadenas (Sección 2.2)
Software que recibe el Excel y devuelve los productos anómalos	Despliegue en producción y soporte posterior

La corrección automática del pedido queda fuera de manera deliberada: decidir por la tienda cuánto debe pedir excede la responsabilidad que el proyecto puede asumir en esta etapa.

3.2. Restricciones, supuestos e incertidumbres

Tabla 14: Lo que se planifica, lo que se valida y lo que se investiga.

Restricción	Supuesto	Incertidumbre
El extracto cubre 11 semanas y no se puede alargar hacia atrás	Que las cantidades son comparables entre líneas del mismo producto	Cuánta de la desviación de una semana de campaña es demanda y cuánta error
No se puede modificar el ERP del cliente	Que el historial de correos alcanza para marcar suficientes casos	Cuántos de los 27 pedidos rectificados son errores y cuántos cambios de necesidad
El proyecto cierra en la semana 12	Que el extracto refleja operación normal y no un período atípico	Con qué frecuencia aparecen errores por debajo del umbral, que hoy no se detectan
Los datos deben anonimizarse para uso académico	Que las alertas se responden dentro de los 30 minutos pactados	Si el patrón se sostiene al ampliar el historial más allá de 11 semanas

El caso más peligroso no es que el sistema deje pasar un error, sino el contrario: que marque un pedido correcto, alguien le crea y recorte una cantidad que la tienda sí necesitaba. Ahí el proyecto deja de ahorrar y empieza a costar, porque un quiebre de stock vale más que el exceso que se buscaba evitar: la venta no ocurre y el reclamo pasa a ser del cliente hacia el proveedor. Los correos revisados muestran que la operación ya conoce ese riesgo y decide en su favor: ante la duda se despacha igual, para no desabastecer.

De ahí salen dos decisiones de diseño que aparecen en todo el documento. El sistema **alerta pero no bloquea**, de modo que ninguna alerta puede reducir un pedido por sí sola. Y la precisión se prioriza sobre la cobertura, aceptando que se escapen errores pequeños antes que marcar pedidos legítimos.

3.3. Qué se va a medir

Tabla 15: Evidencia mínima del proyecto.

Qué mido	Hoy	Con la solución	Cuándo
Pedidos erróneos detectados antes del despacho	Solo los que alguien nota al revisar el archivo	$\geq 80\%$ de los confirmados	Semana 7
Precisión de las alertas	Sin medir	$\geq 70\%$	Semana 7
Alertas emitidas por semana	1 documentada en el extracto	entre 2 y 5	Semana 8
Unidades despachadas de más y detectadas	0	$\geq 80\%$ de las 11 200 identificadas en el extracto	Semana 12
Tiempo entre la carga del archivo y el resultado	Revisión manual de todo el archivo	Menos de un minuto	Semana 10

3.4. Indicadores

Tabla 16: Indicadores de seguimiento.

Indicador	Cómo se calcula	Meta	Frecuencia	Fuente
Precisión	Alertas confirmadas como error / alertas emitidas	$\geq 70\%$	Semanal	Registro de casos marcados
Exhaustividad	Errores detectados / errores confirmados	$\geq 80\%$	Semanal	Registro de casos marcados
Carga de alertas	Alertas emitidas por semana	2 a 5	Semanal	Salida del software
Error del valor esperado	Error absoluto medio frente a la persistencia	< 1	Semanal	Extracto
Unidades marcadas	Exceso en unidades sobre el valor esperado	Seguimiento	Semanal	Salida del software

3.5. Criterios de aceptación del prototipo

Cada criterio está redactado de modo que la respuesta sea verdadero o falso, sin lugar a interpretación, e indica quién lo verifica y con qué evidencia queda demostrado.

Tabla 17: Criterios de aceptación del software.

	Criterio	Quién verifica	Evidencia
1	Dado el archivo exportado del sistema de compras sin modificar, cuando se ejecuta la validación, entonces el software termina sin error y devuelve un Excel con las nueve columnas de la Tabla 11 completas en todas las filas.	Practicante	Archivo de salida de una semana real
2	Dado el caso documentado de la tienda de ATE en la semana del 17 de agosto, cuando se ejecuta la validación sobre ese período, entonces esa línea aparece marcada con nivel alto.	Practicante	Salida del software y correo de alerta original
3	Dadas dos líneas vigentes con la misma tienda, producto, fecha de entrega y cantidad, cuando se ejecuta la validación, entonces ambas quedan marcadas con motivo «duplicidad».	Practicante	Salida sobre los 54 casos identificados en el extracto
4	Dada una serie tienda–producto con menos de k semanas de historial, cuando se evalúa su pedido, entonces aparece en la hoja de no evaluables y no en la de anomalías.	Practicante	Salida del software
5	Dada una semana cuyo factor común supera κ , cuando se ejecuta la validación, entonces el reporte incluye la advertencia de posible campaña.	Practicante	Salida sobre la semana del 27 de julio
6	Dada una tienda con valor esperado positivo y sin ninguna línea en el archivo, cuando se ejecuta la validación, entonces aparece en la hoja de ausencias con las semanas que lleva sin pedir.	Practicante	Salida sobre las 7 series con huecos intermedios
7	Dado el mismo archivo de entrada, cuando se ejecuta la validación dos veces, entonces ambos resultados son idénticos.	Practicante	Comparación de los dos archivos de salida
8	Dado un archivo al que le falta una columna obligatoria, cuando se ejecuta la validación, entonces el software nombra la columna faltante y no procesa.	Practicante	Captura del mensaje de error
9	Dado el archivo de una semana, cuando lo ejecuta alguien ajeno al proyecto sin acompañamiento, entonces obtiene el resultado en menos de un minuto y sin consultar el manual.	Practicante	Acta de la prueba de uso

El criterio 9 es el que suele darse por supuesto y no se escribe. Un software que entrega el resultado correcto pero que solo sabe ejecutar quien lo programó no resuelve el problema de nadie más.

3.6. Viabilidad

Técnica. El volumen es pequeño y los modelos son ligeros. Python, `pandas` y `scikit-learn` bastan en un equipo corriente [12]. El proyecto está dimensionado para el perfil de quien lo ejecuta, una ingeniera industrial con manejo de análisis de datos: no requiere desarrollo web, base de datos ni despliegue en servidores. La interfaz se arma con Streamlit, que convierte un script de análisis en una aplicación con carga de archivos, tablas y gráficos sin escribir código de frontend, y las reglas legibles del árbol se obtienen directamente de `export_text` en `scikit-learn`. La parte propiamente informática del trabajo se reduce así a encadenar herramientas existentes.

Operativa. El área ya hace esta verificación a mano y aplica el mismo criterio de comparar contra el período anterior, así que el sistema no le pide cambiar de forma de pensar. Dos a

cinco alertas semanales caben en la rutina que ya existe y en la ventana de 30 minutos ya pactada.

Económica. No hay licencias ni infraestructura que comprar. El costo es el tiempo de trabajo de la practicante. El exceso que hoy se despacha, unos S/ 3 200 al mes, supera el costo de las horas de validación necesarias para poner el sistema en marcha.

3.7. Matriz de riesgos

Cada riesgo lleva su probabilidad, su impacto y una **señal temprana**: el dato observable que avisa que el riesgo se está materializando, antes de que sea tarde para reaccionar.

Tabla 18: Riesgos del proyecto.

Riesgo	Prob.	Imp.	Señal temprana	Mitigación
No se logran marcar suficientes casos como error	Alta	Alto	Al revisar los veinte candidatos, la mayoría queda en duda	Revisarlos en la semana 4; si no alcanzan, pasar a detección no supervisada y replantear las metas
Sobreajuste del árbol con 176 series y 3 productos	Media	Alto	La precisión en validación cae respecto de la de entrenamiento	Profundidad limitada, validación temporal, agrupar por presentación
La referencia queda inflada porque el historial contiene los errores	Alta	Medio	El valor esperado sube semana a semana sin que suba el consumo	Mediana y MAD en lugar de promedio; recálculo excluyendo lo marcado
Campañas y feriados generan alertas correctas pero inútiles	Media	Alto	Más del 10 % del universo supera el umbral la misma semana	Separar el movimiento de toda la cadena del de cada tienda (Sección 1.7.5)
Muy pocos casos anómalos frente al total	Alta	Medio	La exactitud global supera el 95 % sin detectar nada	Reportar precisión y exhaustividad, nunca exactitud
El software no se adopta y se vuelve a la revisión manual	Media	Alto	En la prueba de la semana 12 alguien ajeno al proyecto no logra ejecutarlo solo	Criterio de aceptación 9; interfaz de un solo paso; formato de salida igual al que ya usan
Cambio en el formato del archivo exportado	Baja	Alto	Aparecen columnas nuevas o renombradas en el extracto	Verificación de columnas al inicio del proceso, con mensaje explícito
Pérdida de acceso a los datos	Baja	Alto	Demora en la entrega del extracto mensual	Copia anonimizada congelada al inicio del proyecto

Los dos primeros riesgos son los que pueden cambiar el resultado del proyecto y no solo retrasarlo. El resto se administra dentro del plan.

3.8. Plan de trabajo

El proyecto lo ejecuta una sola persona de principio a fin. No hay un equipo ni un área que valide entregas intermedias: el resultado se presenta a la jefatura al cierre y a la universidad como entregable académico. El plan está armado en consecuencia, sin pasos que dependan de que responda un tercero.

Las dos primeras semanas ya están hechas. El análisis exploratorio de la Sección 1.7 cubre la limpieza del extracto, el cálculo del valor esperado, la separación del residuo, la detección

de duplicados y de ausencias y la estimación del exceso, y funciona sobre los datos reales. Lo pendiente empieza en la semana 3.

Dos decisiones de orden vale la pena explicar. Las reglas simples se miden antes de entrenar el árbol, no después, para que la comparación no se acomode al resultado: el número contra el que hay que ganar ya está escrito cuando el árbol se entrena. Y el paso 3 va antes que el 4 porque sin casos marcados no hay con qué medir ninguno de los dos.

El paso 10 merece una nota, porque es el que hace que la herramienta mejore con el uso. Cada vez que alguien confirma o descarta una alerta desde la ficha queda un ejemplo marcado, sin trabajo adicional. A las pocas semanas de operación hay más casos revisados que los del paso 3, y el modelo puede reentrenarse con ellos. El etiquetado deja de ser un esfuerzo puntual del inicio y pasa a ser un subproducto del uso.

El hito del paso 5 es el que puede cambiar el rumbo. Si el árbol no le gana a las reglas simples, se deja el árbol y el software se construye igual sobre la regla que haya ganado: cambia cómo se decide qué marcar, no el resto de la herramienta. Que el problema se resuelva con una regla sencilla es un resultado válido y así se reportaría.

4. Retroalimentación incorporada

El asesor planteó delimitar mejor el problema: que el pedido no lo envíe el usuario de tienda sino directamente un software. La idea ataca el error en su origen, porque si nadie digita no hay nada que digitar mal. Se evaluó y se optó por otra delimitación, porque las dos propuestas automatizan cosas de naturaleza distinta y solo una conviene automatizar: el envío del pedido es un proceso de cara al cliente, la detección de errores es un control interno del proveedor.

Tabla 20: Automatizar el pedido frente a automatizar la detección.

	Automatizar el pedido	Automatizar la detección
Dónde corre	En el proceso de compras del cliente	En el proveedor, sobre su propio archivo
Permisos	Compras y TI del cliente	Ninguno
Al replicarlo con otro cliente	Hay que rehacerlo: otro ERP, otro formato, otro calendario	Funciona igual: el dato de entrada es del proveedor
Si se equivoca	El pedido errado lo emitió el proveedor	Se pierde una alerta; el pedido sigue siendo del cliente
Sobre la relación	Elimina el contacto semanal	Le da contenido

Automatizar el flujo con el cliente tiene tres costos. El primero es que el esfuerzo no se acumula: el pedido cambia de cliente a cliente y con la temporada, de modo que habría que construir y mantener una integración por cada cadena, sin reutilizar nada con la siguiente. El segundo es que ese pedido semanal es la línea directa con el cliente, el momento en que el proveedor se entera de lo que la tienda cree que va a necesitar; ahí aparecen las señales de que algo cambió, desde una caída sostenida hasta el reparto del volumen con otro proveedor. Un sistema que genera los pedidos por su cuenta deja de recibir esa señal y se enteraría tarde, cuando el volumen ya bajó. El tercero es un traslado de responsabilidad: hoy, si la cantidad está mal, el error es del cliente y el proveedor tiene con qué sustentarlo; si el proveedor emitiera los pedidos, cualquier sobre-despacho pasaría a ser error propio, con la discusión comercial y la devolución a cuenta suya.

El control interno no tiene ninguno de esos costos. Corre sobre datos que el proveedor ya tiene, sirve igual para cualquier cliente que emita pedidos, y no sustituye la conversación

Tabla 19: Pasos del proyecto, su producto, la semana en que cierran y la herramienta con que se hacen.

#	Paso	Qué queda al terminarlo	Sem.	Con qué
<i>Hecho</i>				
1	Limpiar el dataset y explorarlo. Quitar las líneas anuladas, revisar formatos, armar las series semanales y medir el problema	Archivo limpio, el programa que lo limpia y las cifras de la Sección 1.7	1–2	pandas
<i>Modelo</i>				
2	Armar la tabla de señales. Para cada pedido, calcular las ocho señales que después mirará el modelo	Tabla con una fila por pedido y sus ocho señales	3–4	pandas
3	Marcar los casos que fueron error. Revisar los candidatos uno por uno apoyándose en el historial de correos de alerta y en las 27 líneas que alguien rectificó	Lista de casos marcados como error o como normales	4–5	Excel
4	Probar primero las reglas simples. Medir tres métodos sencillos para saber contra qué hay que ganarle	Resultado de las tres reglas	5–6	pandas, sklearn
5	Entrenar el árbol y compararlo con las reglas del paso 4	Comparación semana por semana. Hito: se decide si el árbol se justifica	6–7	sklearn
6	Decidir a partir de cuánto se avisa, buscando el punto donde el número de alertas por semana sigue siendo revisable	Umbral fijado y curva umbral-alertas	8	pandas
7	Escribir el motivo en palabras. Convertir lo que decide el modelo en una frase que el cliente entienda	Texto del motivo para cada alerta	8	export_text
<i>Software</i>				
8	Motor de cálculo. Una función que recibe la ruta del Excel y devuelve la tabla de alertas	Programa que ya funciona, aunque sin pantallas	9	pandas
9	Pantalla de carga y tabla de alertas. Arrastrar el archivo, avisar qué falta si algo no está, y mostrar el resultado con filtros por tienda, producto y nivel	Aplicación usable de punta a punta	10	Streamlit
10	Ficha de detalle y registro de decisiones. Gráfico de las últimas semanas, las otras dos presentaciones, la regla que disparó la alerta, y botones para confirmar o descartar	Ficha por alerta e historial de decisiones	11	Streamlit
11	Ausencias, aviso de campaña y descarga del Excel de salida	Archivo de salida completo	11	Streamlit
<i>Cierre</i>				
12	Probarlo sobre una semana real, ajustar y documentar	Manual de uso, informe final y resultados	12	—

sino que le da contenido: en lugar de un correo que dice «su pedido parece alto», el área envía la cantidad solicitada, la esperada según el historial de esa tienda y el motivo. Eso fideliza más que hacerle el pedido, porque demuestra que alguien está mirando sus números. La regla que ordena la decisión es automatizar donde el proceso es estable y propio, y dejarlo fuera donde es variable y compartido.

La observación sí amplió el alcance en un punto. Si el software estima la demanda esperada de cada tienda, también puede detectar la ausencia de pedido: una tienda que debía pedir y no pidió. Ese caso no estaba contemplado y se incorporó según la Sección 2.7.

5. Estado actual del proyecto

El proyecto se encuentra en la **etapa de análisis**. Lo que este entregable presenta es el problema delimitado, medido y con un método de detección probado sobre los datos históricos. La construcción del software todavía no ha empezado: corresponde a los pasos 8 a 12 del plan de trabajo.

Lo que ya está resuelto es lo siguiente. El problema dejó de ser una queja y pasó a tener cifras: 54 casos de cantidad repetida, 17 desviaciones relevantes en nueve semanas y alrededor de S/ 3 200 mensuales en producto despachado por encima de lo esperado. El criterio de detección se probó contra un caso que la operación ya había identificado por su cuenta, el de la tienda de ATE, y lo reencontró sin ayuda. Y el principal riesgo del método, confundir un feriado con un error, quedó resuelto dentro del propio cálculo mediante la separación entre el movimiento común de la cadena y el de cada tienda, sin depender de que alguien recuerde qué pasó una semana determinada.

Lo que falta es lo que va de la semana 3 en adelante. El árbol de decisión aún no está entrenado, los cinco parámetros de la Tabla 6 tienen valores iniciales pero no ajustados con validación temporal, y el software existe como especificación y no como programa. Las métricas comprometidas en la Tabla 16 son metas y todavía no resultados.

Referencias

- [1] Charu C. Aggarwal. *Outlier Analysis*. Springer, 2nd edition, 2017.
- [2] Scott W. Ambler and Mark Lines. *'Elija su WoW! Un enfoque de Disciplined Agile para optimizar su forma de trabajar*. Project Management Institute, 2. extsuperscripta edition, 2023.
- [3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth and Brooks/Cole, Monterey, CA, 1984.
- [4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009. Artículo 15.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [6] Victoria J. Hodge and Jim Austin. A survey of outlier detection methodologies. *Artificial Intelligence Review*, 22(2):85–126, 2004.

- [7] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. <https://otexts.com/fpp3/>.
- [8] Kaoru Ishikawa. *Guide to Quality Control*. Asian Productivity Organization, Tokio, 2nd edition, 1986.
- [9] Hau L. Lee, V. Padmanabhan, and Seungjin Whang. Information distortion in a supply chain: The bullwhip effect. *Management Science*, 43(4):546–558, 1997.
- [10] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422. IEEE, 2008.
- [11] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774, 2017.
- [12] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [13] Project Management Institute. *The Standard for Project Management and A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, Newtown Square, PA, 8th edition, 2025.
- [14] J. Ross Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA, 1993.
- [15] Rüdiger Wirth and Jochen Hipp. CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining*, pages 29–39, 2000.